Field Hours

CSCE 670 Information Storage and Retrieval

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

The inverted index

You do not scan every document for every query once the collection is large. You build an inverted index: each term maps to a postings list of document identifiers. You tokenize each document and emit pairs of term and doc id, then sort and group by term. In a collection of 4 documents, cat might post 2 identifiers. A term that appears once posts a list of length 1.

You keep each postings list sorted by document identifier so two lists can be merged in one pass. You also store document frequency, the length of that list. A term in 3 of 4 documents is less selective than a term in 1 of 4. The index stores locations, not the original sentence. If phrase queries matter, store a position too: cat at position 2 beside dog at position 3 is a different fact from those words far apart.

Tokens become one postings list.

Tokenize each document -> Emit term and doc id -> Sort and group by term -> Store the sorted postings

Index four short documents

1. Documents 1 and 3 contain cat. Documents 2 and 4 do not. You emit the pairs (cat, 1) and (cat, 3).

2. After sorting, the postings for cat are 1, 3. The document frequency is 2.

3. A second term, dog, occurs only in document 4, so its list has length 1 and its document frequency is 1.

Problem 1. In a collection of 4 documents, cat occurs in 2 of them. What is stored for cat, and what is its document frequency?

Answer. A sorted postings list of those 2 document identifiers, and a document frequency of 2. The index does not store the original sentences.

Problem 2. cat is at position 2 and dog is at position 9 in the same document. Why is that not the same fact as cat at 2 beside dog at 3?

Answer. Positions 2 and 3 are adjacent, which can support a phrase. Positions 2 and 9 are only a bag of words. If you need phrases, the postings have to store positions, not only document identifiers.

Lecture 2 · 90 minutes

Boolean queries on postings

A Boolean query combines terms with AND, OR, and NOT. AND is the intersection of two sorted postings lists.

You walk the lists with two pointers and emit an identifier only when both pointers see it, then you advance the pointer that is behind. OR is the union, and you emit every identifier once. NOT against a corpus of 10000 documents turns a list of length 2 into a huge complement you rarely want.

The order of operations changes the work. Intersect the shorter list first. If cat posts 3 documents and animal posts 400, you intersect cat with the next term before you touch the long list, when the Boolean structure allows it. Conjunctive queries are the common case in web search even when the user typed no AND. Lists 1, 4, 9, 12 and 4, 7, 12 intersect at 4 and 12.

Intersect two sorted lists.

1. Compare the two heads

2. Emit an id only on equality

3. Advance the smaller head

4. Stop at the end of either list

Intersect 1, 4, 9, 12 with 4, 7, 12

1. Heads are 1 and 4. 1 is smaller, so you advance the first list. Heads are now 4 and 4.

2. They match, so you emit 4 and advance both. Next heads are 9 and 7. Advance the second list to 12.

3. 9 is behind 12, so you advance to 12 and 12, emit 12, and stop. The intersection has size 2.

Problem 1. What is the intersection of postings 1, 4, 9, 12 and 4, 7, 12, and why do you not emit 7?

Answer. The intersection is 4 and 12. 7 appears in only one list. AND emits an identifier only when both pointers see it.

Problem 2. A term's postings have length 2, and the corpus has 10000 documents. Why is NOT on that term a poor query by itself?

Answer. The complement has 9998 documents. NOT turns a short list into almost the whole corpus, which is rarely the set you meant to retrieve.

Lecture 3 · 90 minutes

Ranking with tf-idf

Boolean retrieval returns a set, and ranked retrieval returns an order. You weight a term by its frequency in the document times its inverse document frequency. Use log base 2. If N is 4 and the term appears in 2 documents, inverse document frequency is log2 of 4/2, which equals 1. A document with term frequency 3 then has weight 3. A term that appears in all 4 documents has weight contribution 0, because log2 of 1 is 0.

A multi-term query scores a document by summing weights of query terms that occur in it. You ignore document terms the query did not ask for. With idf of cat equal to 1 and idf of dog equal to 2, a document that has cat twice and dog never scores 2. A document that has each term once scores 3, because the rare term contributes 2. You sort by that sum and put the higher score first.

Two documents, one query.

Document 1

cat twice, no dog

Score is 2

Ranks second

Document 2

cat once, dog once

Score is 3

Ranks first

Score cat and dog

1. idf(cat) is 1 and idf(dog) is 2. The query contains both terms.

2. Document 1 has term frequencies 2 and 0, so the score is 2 times 1, plus 0, which is 2.

3. Document 2 has term frequencies 1 and 1, so the score is 1 times 1, plus 1 times 2, which is 3. You rank document 2 first.

Problem 1. N is 4 and a term appears in all 4 documents. What is its inverse document frequency, and can it change the ranking?

Answer. log2 of 4/4 is log2 of 1, which is 0. Every document gets weight 0 from that term, so it cannot change the order.

Problem 2. idf(cat) = 1 and idf(dog) = 2. Document A has cat three times and no dog. Document B has each term once. Which ranks first, and what are the scores?

Answer. A scores 3 times 1 = 3. B scores 1 times 1 + 1 times 2 = 3. They tie at 3. Neither ranks strictly first. The rare term's extra 2 on B matches A's extra cat counts.

Lecture 4 · 90 minutes

Precision, recall, and a cutoff

A ranked list is not a success until you compare it with judgments. Precision is the fraction of retrieved documents that are relevant. Recall is the fraction of relevant documents that you retrieved. If you return 10 documents and 4 of them are relevant, precision is 4/10, which is 0.4. If the collection holds 5 relevant documents, recall is 4/5, which is 0.8. The two numbers move differently when you lengthen the list: recall can rise while precision falls.

You must fix a cutoff, because a ranked list can be the whole collection. Precision at 10 uses the first 10 hits only. If 4 relevant hits sit at ranks 11 to 14, recall at 10 is 0. Average precision averages the precision at each relevant hit's rank, over all relevant documents, using 0 when a hit is missing. You write the cutoff beside precision or two runs cannot be compared.

One list, two different fractions.

10 retrieved documents

4 of those are relevant

5 relevant documents exist

Precision 0.4 and recall 0.8

Average precision for two hits

1. Two documents are relevant, and both are retrieved, at ranks 1 and 3. No relevant document is missing.

2. Precision at rank 1 is 1/1 = 1. Precision at rank 3 is 2/3.

3. Average precision is (1 + 2/3) / 2 = (5/3) / 2 = 5/6. You divide by the 2 relevant documents, not by the length of the list.

Problem 1. You return 10 documents, 4 are relevant, and the collection has 5 relevant documents. What are precision and recall?

Answer. Precision is 4/10 = 0.4. Recall is 4/5 = 0.8. You need both, because lengthening the list can raise recall while precision falls.

Problem 2. The only 2 relevant documents sit at ranks 1 and 3. What is the average precision?

Answer. 5/6. Precision at those ranks is 1 and 2/3. Their sum is 5/3, and you divide by 2 relevant documents, which gives 5/6.

Lecture 5 · 90 minutes

Gaps and variable bytes

Document identifiers in a postings list are sorted, so you can store gaps instead of the raw identifiers. The first gap is the first identifier, and each later gap is the difference from the previous identifier. The list 5, 6, 9, 20 becomes the gaps 5, 1, 3, 11. The gaps sum to 20, which is the last identifier, so you can rebuild the list by a running total.

You encode each gap in as few bytes as the code allows. Seven bits of each byte hold data, and the eighth bit says whether another byte follows. A gap of 127 or less fits in 1 byte. A gap of 200 needs 2 bytes, because 200 is greater than 127. Three gaps, 4, 10, and 200, therefore cost 1 + 1 + 2 = 4 bytes. Storing the same three raw identifiers as 4-byte integers would cost 12 bytes.

Identifiers become short gaps.

Sort the document ids -> Store differences as gaps -> Encode each gap in bytes -> Rebuild by a running sum

Encode three gaps

1. The gaps are 4, 10, and 200. Values through 127 fit in 1 byte, so 4 and 10 take 1 byte each.

2. 200 is greater than 127, so it takes 2 bytes. The three gaps cost 4 bytes.

3. Three raw identifiers stored as 4-byte integers would cost 12 bytes. The gap encoding is smaller on this list.

Problem 1. Postings are 5, 6, 9, 20. What gaps do you store, and what is their sum?

Answer. Gaps 5, 1, 3, and 11. They sum to 20, the last identifier. A running total rebuilds 5, then 6, then 9, then 20.

Problem 2. How many bytes do variable-byte codes use for gaps 4, 10, and 200, and how many bytes do three 4-byte integers use?

Answer. 4 bytes for the gaps: 1 plus 1 plus 2, because 200 exceeds 127. Three 4-byte integers use 12 bytes.

Lecture 6 · 90 minutes

Walking postings for a query

Term-at-a-time scoring finishes one query term's postings before it reads the next term. Document-at-a-time scoring walks the lists in parallel, in document-identifier order, and keeps a partial score only for the document it is on. With 3 query terms you either make 3 passes over accumulators or you advance 3 pointers together. Document-at-a-time uses less memory for accumulators when you only keep the current candidates, which matters when the collection has millions of documents and the query is short.

A skip pointer jumps forward in a postings list when the other list is already past this block. Place a skip every 4 postings. A list of 16 identifiers then carries 4 skips. During AND, if the other pointer is beyond the skip target, you jump rather than compare the next 4 identifiers. The intersection does not change. You only change the number of comparisons required to produce it.

Two ways to score three terms.

Term at a time

Finish one list first

3 passes over scores

Accumulators stay large

Document at a time

Walk lists together

3 pointers stay in step

Score the current doc

Four skips on a list of 16

1. The postings list has 16 identifiers. You place a skip every 4 postings, so there are 4 skips.

2. During AND, the other list's head is past the end of the current block. You follow the skip instead of

comparing those 4 identifiers.

3. The emitted intersection is unchanged. You saved comparisons, not a different answer.

Problem 1. A query has 3 terms. What is the difference between term-at-a-time and document-at-a-time in how you touch those lists?

Answer. Term-at-a-time finishes one term's postings before the next, which is 3 passes. Document-at-a-time advances 3 pointers together in document-identifier order and scores the current document.

Problem 2. A list has 16 postings and a skip every 4. How many skips is that, and does a skip change the AND result?

Answer. 4 skips. The intersection does not change. A skip only avoids comparisons inside a block the other pointer has already passed.

Lecture 7 · 90 minutes

One step of PageRank

Term weights ignore links. PageRank scores a page from the pages that point at it. The update is (1 - d) / N plus d times the sum of each in-neighbor's score divided by that neighbor's out-degree. With damping d = 1/2 and N = 3 pages, the teleport piece (1 - d) / N equals 1/6. You start from a uniform score of 1/3 on each page. A page still receives 1/6 even if nobody links to it.

Take pages A, B, and C. A links to B and C, B links only to C, and C links only to A. One update from uniform 1/3 scores gives A = 1/3, B = 1/4, and C = 5/12. C ranks first after that step because A and B both point at C, while A must split its credit across 2 out-links. You have done 1 iteration, not a fixed point. Out-degree is what splits a page's credit.

Credit into C on one step.

1. Start each page at 1/3

2. A splits across 2 links

3. B sends all its score to C

4. C becomes 5/12

Update A, B, and C once

1. Teleport is 1/6 and d is 1/2. A receives only C: 1/6 + (1/2) * (1/3) = 1/3.

2. B receives half of A, because A has 2 out-links: 1/6 + (1/2) * ((1/3) / 2) = 1/6 + 1/12 = 1/4.

3. C receives half of A plus all of B: 1/6 + (1/2) * ((1/3) / 2 + 1/3) = 1/6 + 1/4 = 5/12. The three scores sum to 1.

Problem 1. After one update from uniform 1/3, with d = 1/2, what are the scores of A, B, and C, and which page ranks first?

Answer. A is 1/3, B is 1/4, and C is 5/12. C ranks first. Those three fractions sum to 1, and this is a

single iteration, not a fixed point.

Problem 2. Why does A send only half of its score toward C, while B sends all of its score toward C?

Answer. A has 2 out-links, so its credit is split. B has 1 out-link, so C receives B's whole score. Out-degree is the split.

Lecture 8 · 90 minutes

Index, score, and measure

You can now build a small retrieval system and say whether it worked. Tokenize documents into a sorted inverted index, answer AND by intersecting postings, and rank a free-text query with tf-idf using a base-2 log. Compress the postings as gaps if you store them, and walk them document-at-a-time when the query has several terms. Judge the top 10 against a set of relevant ids and report precision and recall at that cutoff.

The pieces have to use the same document identifiers. A gap list that decodes to 5, 6, 9, 20 is the same postings the intersect routine expects, not a different numbering. A tf-idf score that ranks document 2 above document 1 is not a precision. If your top 2 contains 1 relevant document and there are 4 relevant in total, precision at 2 is 1/2 and recall at 2 is 1/4.

From bytes to a measured list.

Build sorted postings -> Score with tf-idf -> Cut the list at 10 -> Report precision and recall

A score is not yet a precision

1. Document 2 scores 3 and document 1 scores 2, so the ranked order is document 2, then document 1.

2. Suppose document 2 is relevant and document 1 is not, and 4 documents in the collection are relevant.

3. Precision at 2 is 1/2. Recall at 2 is 1/4. Reporting only the scores 3 and 2 never states either fraction.

Problem 1. Gaps decode to document ids 5, 6, 9, 20, and a second list is 4, 12. You also have tf-idf scores that rank document 2 above document 1. Which of those ids can the intersection use, and why are the scores not a precision?

Answer. The intersection can use only the decoded ids, so it looks for shared values among 5, 6, 9, 20 and 4, 12, which is empty. Document 2 and document 1 are a different numbering unless you have said they are those ids. The scores 3 and 2 are not a precision, because precision needs relevance judgments and a cutoff.

Problem 2. Your top 2 contains 1 relevant document, and 4 documents in the collection are relevant. What are precision at 2 and recall at 2?

Answer. Precision at 2 is 1/2. Recall at 2 is 1/4. You report both. Either fraction alone hides what the cutoff did.